Lecture 23: Broadcasting

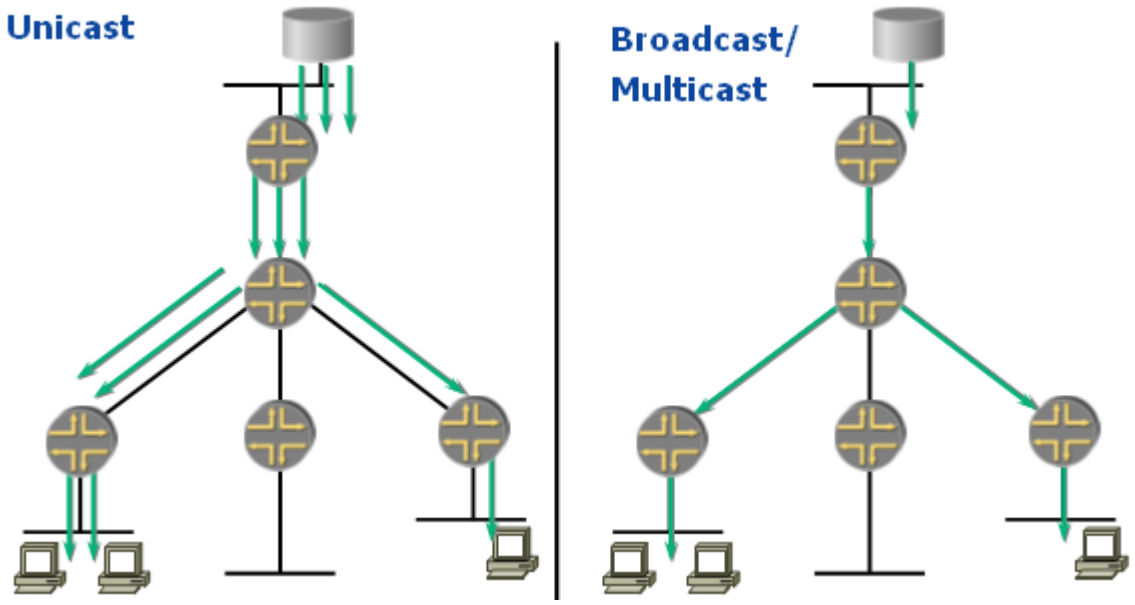
Objectives:

- Learn about broadcasting and its advantages over unicasting
- Learn how to write applications that use broadcasting

1. Introduction

So far all our multi-client programs were designed to send a separate message for each client - **unicasting**.

In this lecture and the next, we shall learn two ways of sending a single message to multiple clients at the same time, namely, **broadcasting** and **multicasting**.



Clearly, broadcasting and multicasting will decrease the amount of network bandwidth used by an application.

2. What is broadcasting?

IP broadcasting is used by network devices to send a single packet of information to every device on a network. [one-to-all]

This requires the use of UDP since TCP requires connections between communicating devices.

IP address format allows two types of broadcast addresses: local broadcast addresses and global broadcast addresses.

Local Broadcast Address:

The local broadcast address is used to send broadcast address to all devices in a particular subnet.

Recall that an IP address consists of two components, the network part and the host part.

The local broadcast IP address consists of the network address of a subnet together with all ones (255 in decimal) in the host part.

For example, for a class B network: 191.168.0.0, using the subnet mask of 255.255.0.0, the local broadcast address is 191.168.255.255.

If the subnet is further subdivided using subnet mask 255.255.255.0, then each subnet will have its own local broadcast address.

For example, the subnet 191.168.254.0 will have the local broadcast address of 191.168.254.255.

Global Broadcast Address:

The global broadcast address was originally intended to allow a device to send packets to all devices on an inter-network. It uses all ones in an IP address: 255.255.255.255.

The reality of the Internet (its popularity and security issues) dictated that global broadcasting is not feasible due to the possibility of using it to crash the Internet.

Thus, routers do not send global IP broadcast to other networks unless specifically configured to do so, which is practically never. Instead, they silently ignore global broadcast messages.

Implementing Broadcasting Systems in C#

By default, sockets are not allowed to send broadcast messages – doing so will cause a Socket Exception to be thrown.

To send broadcast packets, the broadcast socket option must be set on the socket using the ***SetSocketOption*** method as shown below:

```
Socket socket = new  Socket(AddressFamily.InterNetwork,
                          SocketType.Dgram, ProtocolType.Udp);
socket.SetSocketOption(SocketOptionLevel.Socket,
          SocketOptionName.Broadcast, 1);  //1 indicates true
```

After the socket option is set, you specify broadcast address and port number to use for the broadcast as shown below:

```
IPEndPoint endPoint=new IPEndPoint(IPAddress.Broadcast, 9090);
byte[] data = Encoding.ASCII.GetBytes("test message");
socket.Send(data, endPoint);
```

The following is a complete example showing how to send broadcast messages:

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;

public class SimpleBroadcastSender
{
   public static void Main()
   {
     Socket sock = new Socket(AddressFamily.InterNetwork,
SocketType.Dgram, ProtocolType.Udp);
     sock.SetSocketOption(SocketOptionLevel.Socket,
SocketOptionName.Broadcast, 1);
     IPEndPoint endPoint = new IPEndPoint(IPAddress.Broadcast, 9090);

     String message;
     while (true) {
        Console.Write("Enter message to broadcast: ");
        message = Console.ReadLine();

        if (message=="")
           break;

        byte[] data = Encoding.ASCII.GetBytes(message);
        sock.SendTo(data, endPoint);
     }
     sock.Close();
   }
}
```

No special socket options are required to receive broadcast packets. The only requirement is that the application should be listening to the port through which the broadcast packets are being sent.

The following example can be used to receive broadcast messages sent by the SimpleBrodcastSender example shown above.

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;

public class SimpleBroadcastReceiver
{
   public static void Main()
   {
      Socket sock = new Socket(AddressFamily.InterNetwork,
                      SocketType.Dgram, ProtocolType.Udp);
      IPEndPoint localEP = new IPEndPoint(IPAddress.Any, 9090);
      sock.Bind(localEP);

      EndPoint remoteEP = new IPEndPoint(IPAddress.Any, 0);
      Console.WriteLine("Ready to receive...");

      byte[] data;
      int size;
      string message;

      while (true) {
        data = new byte[1024];
        size = sock.ReceiveFrom(data, ref remoteEP);
        message = Encoding.ASCII.GetString(data, 0, size);
        Console.WriteLine("received: {0}  from: {1}",
                                message, remoteEP.ToString());
      }
   }
}
```

Broadcast Chat System:

In lab06, we developed a chat system using UDP, where clients send their messages to a server, which then broadcast the message to all known clients.

There were a number of problems with that system:

- The client must send a message to the server before it can be recognized as a known client.

- The same message is repeatedly sent individually to each client – thus, wasting bandwidth.

- For each message sent, the server has to search its database to check if the client is known – can be an expensive process as the number of clients increase.

The following example shows a broadcast chat system that does not have the above problems.

In this system, no server is required at all and no need to maintain list of clients. Instead, the system sends packets to the broadcast address.

It also listens for messages on the port being used for the broadcast so that it can both send and receive broadcast messages.

```
using System;
using System.Windows.Forms;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Threading;
using System.ComponentModel;

public class BroadcastChatSystem  : System.Windows.Forms.Form {
   private System.Windows.Forms.Label label3;
   private System.Windows.Forms.Label label2;
   private System.Windows.Forms.GroupBox groupBox2;
   private System.Windows.Forms.TextBox inBox;
   private System.Windows.Forms.TextBox portBox;
   private System.Windows.Forms.TextBox outBox;
   private System.Windows.Forms.Button sendBt;
   private System.Windows.Forms.GroupBox groupBox;
   private System.Windows.Forms.TextBox nameBox;

   private Socket sendSocket, receiveSocket;
   private EndPoint remoteEP, localEP, broadcastEP;
   private Thread receiveHandler;

   public BroadcastChatSystem ()
   {
      InitializeComponent();
      remoteEP = new IPEndPoint(IPAddress.Any, 0);
      localEP = new IPEndPoint(IPAddress.Any,
                               int.Parse(portBox.Text));
      receiveSocket = new Socket(AddressFamily.InterNetwork,
                                 SocketType.Dgram, ProtocolType.Udp);
      receiveSocket.Bind(localEP);
```

```
      broadcastEP = new IPEndPoint(IPAddress.Broadcast,
                                   int.Parse(portBox.Text));
      sendSocket = new Socket(AddressFamily.InterNetwork,
                              SocketType.Dgram, ProtocolType.Udp);

      receiveHandler = new Thread(new ThreadStart(ReceiveData));
      receiveHandler.IsBackground = true;
      receiveHandler.Start();
   }

   void InitializeComponent() {
        //deleted
   }

   void OnSend(object sender, System.EventArgs e)  {
      byte[] data = Encoding.ASCII.GetBytes(nameBox.Text+
                                            ": "+outBox.Text);
      sendSocket.SendTo(data, data.Length, SocketFlags.None,
                        broadcastEP);
      outBox.Text = "";
   }

   void ReceiveData() {
      while (true) {
         byte[] data = new byte[2048];
         int recv = receiveSocket.ReceiveFrom(data, SocketFlags.None,
                                              ref remoteEP);
         inBox.Text += Encoding.ASCII.GetString(data, 0, recv)+"";
      }
   }

   protected override void OnClosing (CancelEventArgs e)  {
      base.OnClosing (e);
      receiveHandler.Abort();
      sendSocket.Close();
      receiveSocket.Close();
   }

   public static void Main() {
      Application.Run(new BroadcastChatSystem ());
   }
}
```

Notice that because this application binds its local end point to a particular port number, we cannot run two instances of it on the same machine. Thus, to test it, we need to run it on two different machines.

Alternatively, we can run it together with the **SimpleBroadcastSender** example on the same machine to test.